

ESZTERHÁZY KÁROLY KATOLIKUS EGYETEM

Magasszintű programozási nyelvek II.

2024/2025 tavaszi félév

Programtervező informatikus

2. zárthelyi dolgozat

Általános információk

Töltse le a mellékelt programot vagy hozzon létre *.NET* projectet a **C:\temp** mappába! Mindkét esetben a program (mappa) neve legyen az Ön **neptunkódja**! A kijelölt feladatokat ebben implementálja C# nyelven, majd munkája végétével ugyanilyen néven tömörítse a programot és töltsse föl az alábbi címre!

<https://oc.uni-eszterhazy.hu/s/wcUXZ0UsrHSqTdJ>

A kicsomagolás után nem beazonosítható dolgozatok automatikusan elégtelenek, ezek újraírására nincs lehetőség!

A dolgozat megírása során semmilyen segédanyagot, órai feladatot nem használhat!

A dolgozat megírása során semmilyen kommunikációs csatorna (levelezőrendszer, chat, stb. . .) nem lehet nyitva és nem léphet kapcsolatba senkivel.

Bármilyen a fentiekre utaló magatartás esetén azonnal elégtelen a zh eredménye. Ennek újraírására nincsen lehetőség!

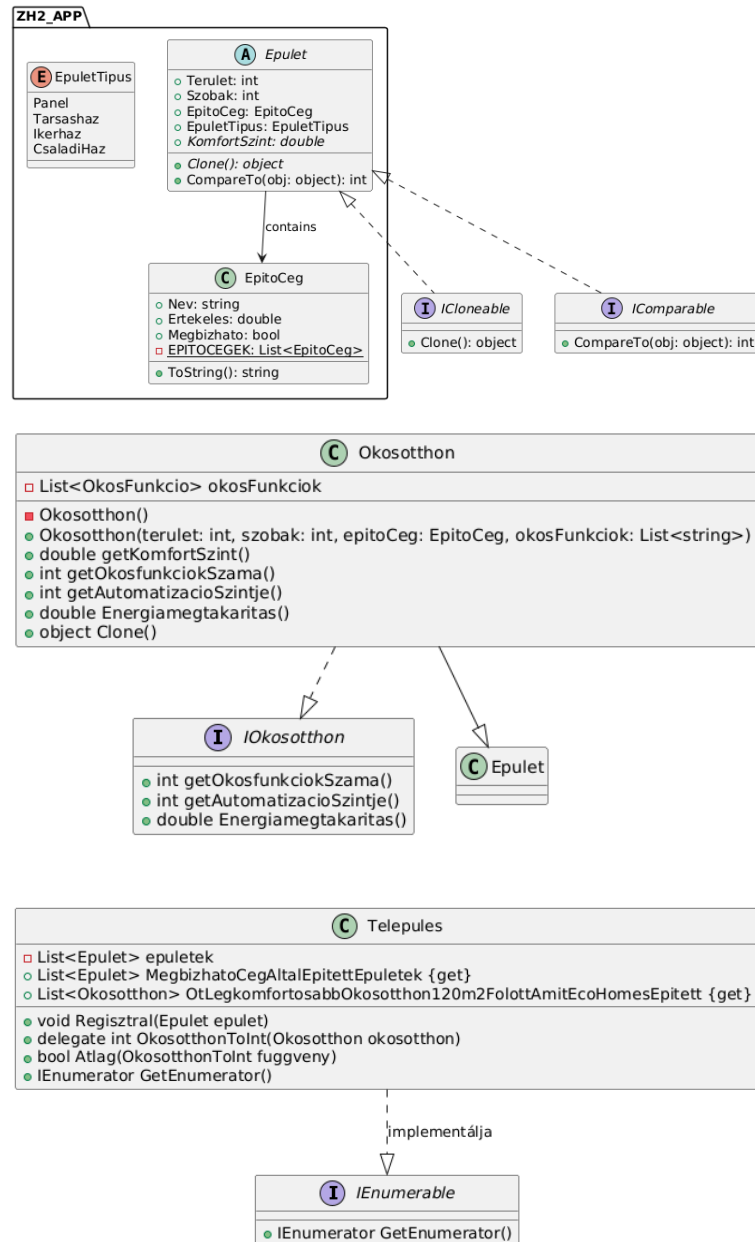
Nem kommunikálhat semmilyen chatbot-tal!

Törekedjen rész megoldásokra!

A második zárthelyi dolgozat fókuszában az osztálystruktúra és a program összetettsége áll. Éppen ezért ahol nem fejtjük ki, hogy egy adatot milyen módon kell validálni, ott elég, ha egy ellenőrzések nélküli property szignatúrát készít!

Feladatkiírás

A zárthelyi dolgozatban egy településen található épületek és okosotthonok adatait feldolgozó alkalmazás alapjait kell elkészítenie! A program alapját képező struktúrát megtekintheti az alábbi osztálydiagramon!



A program alapját az **Epulet** osztály szolgáltatja, mely sokféle épület alapja lehet, Önnek csak az **Okosotthon** gyermekosztály implementációjával kell foglalkoznia! Az épületeket építő cégeket az **EpitoCeg** osztály írja le. Az épületek gyűjteményét a **Telepules** konténerosztály kezeli.

DLL + Console alkalmazás

A feladatot úgy készítse el, hogy az enumok, interfészek és osztályok mind egy DLL-be legyenek implementálva, majd ezt egy másik konzolos alkalmazásban használja fel a főprogramban!

Ha nem biztos magában, inkább hagyja, ez csak 2 pontot ér! Ebben az esetben inkább írjon mindent egyetlen konzolalkalmazásba!

Csatolt forrásfájlok

Annak érdekében, hogy ne kelljen mindent teljesen előlről megírnia, mellékelten talál egy solutiont. Azt, hogy a tartalmát átmásolja egy DLL-be, vagy adott alkalommal átmásolja ebből a szükséges kódrészleteket a saját programjába, vagy éppen végig ebben dolgozik, Önre bízunk!

Javasoljuk, hogy szánjon 2 – 3 percet a programfájlok átnézésére!

Interfészek

A feladatban egyetlen saját interfészt kell létrehoznia!

IOKOSOTTHON

Hozzon létre egy interfészt **IOkosotthon** néven, és implementálja benne az alábbi metódusok és propertyk szignatúráit!

- **OkosfunkciokSzama** nevű egész típussal visszatérő, csak **getter**rel rendelkező property.
- **AutomatizacioSintje** nevű egész típussal visszatérő, csak **getter**rel rendelkező property.
- **Energiamegtakaritas** nevű **valós értékkel visszatérő**, paraméter nélküli függvény.

Osztályhierarchia

EPITOCEG

Nyissa meg a mellékelt **Epitoceg** osztályt, és végezze el az alábbi feladatokat!

- Fejtse ki a **Megbízható** nevű logikai értékű propertyt! Az osztályban talál egy statikus listát, mely a gyártók neveihez társítja, hogy azoknak mennyi az értékelésük. Egy építőcég megbízható, ha az értékelése nagyobb, mint 4.0!
 - Amennyiben nem találja meg a gyártót, úgy dobjon saját kivételt, melynek neve **EpitoCegNemLetezikException**. Ennek a konstruktora paraméterben kérje be a nem talált gyártó nevét és mentse el!

EPULET

Nyissa meg a mellékelt **Epulet** osztályt, és végezze el az alábbi feladatokat!

Propertyk és metódusok

- Fejtse ki a **Terulet** property-t! Amennyiben a terület kisebb, mint 30, úgy dobjon saját kivételt **TeruletAlacsonyException** néven!
- Fejtse ki a **Szobak** property-t! Amennyiben a szobák száma kisebb, mint 1, vagy nagyobb, mint 15, úgy dobjon saját kivételt **SzobakSzamaNemErvenyesException** néven! A kivétel konstruktorában kérje be a rossz értéket, amely a kivételt okozta és tárolja el azt úgy, hogy kívülről ne legyen írható, csak olvasható!
- Hozzon létre egy valós értékkel visszatérő, csak getterrel rendelkező **KomfortSzint** nevű **absztrakt** propertyt!
- Írja felül az **Equals()** metódust! Két épület azonos, ha megegyezik az építőcégük neve, az épület típusa, a területük és a komfortszintjük!
- Írja felül a **ToString()** metódust és jelenítse meg az összes property értékét!

Interfészek

- Implementálja az **ICloneable** interfészt! Lévén az osztály absztrakt, a **Clone()** metódust tegye absztrakttá!
- **Extra:** Implementálja az **IComparable** interfészt! Két épület esetén az a nagyobb, amelynek nagyobb a komfortszintje! Azonos komfortszint esetén a nagyobb területű épület a nagyobb!

OKOSOTTHON

Nyissa meg az **Okosotthon** osztályt, és végezze el az alábbi feladatokat!

- Jelölje meg ősként az **Epulet** osztályt!

Konstruktor

- Egészítse ki a konstruktort úgy, hogy az épület típusán kívül minden mást is kérjen még be! Az ősosztály konstruktorának hívásakor az épület típusát automatikusan állítsa **CsaladiHaz**-ra!

Interfészek

- Implementálja az **IOkosotthon** interfészt!
- Fejtse ki az **OkosfunkciokSzama** csak getterrel rendelkező propertyt! Ennek az értéke az *okosFunkciok* lista elemeinek a száma. Figyeljen rá, hogy a lista többször tartalmazhatja ugyanazokat az értékeket (pl.: Kamera), de ezeket csak egyszer szabad figyelembe vennie a számításnál! (*Segítség: Gyűjtse ki egy listába az értékeket ismétlődés nélkül!*)
- Fejtse ki az **AutomatizacioSzingje** csak getterrel rendelkező propertyt! Ennek értéke az alábbi képlettel számolható.

$$\frac{\text{Okosfunkciók száma}}{6} \cdot 100$$

- Fejtse ki az **Energiamegtakaritas** függvényt! Ennek értéke az alábbi képlettel számolható.

$$\frac{\text{Energiahatékonyságot növelő komponensek száma}}{|100 - \text{terület}|},$$

ahol az *Energiahatékonyságot növelő komponensek száma* az *okosfunkciok* paraméter azon elemeinek száma, melyek nem **Kamera** és nem **Multimedia**!

Propertyk és metódusok

- Fejtse ki a **Komfortszint** propertyt! Ehhez használja az alábbi képletet!

$$\frac{\text{AutomatizacioSzingje} \cdot \text{Energiamegtakaritas}}{5}$$

- Fejtse ki a **Clone()** metódust!
 - Nem használhatja a *MemberwiseClone()* metódust!
 - Az osztályban direkt erre a célra talál egy paraméter nélküli privát konstruktort (ezzel könnyebb dolgoznia).
 - Figyeljen arra, hogy a referencia típusú változók (építő cég, okosfunkciók) esetén ne csak a referenciát adja át, hanem hozzon létre új objektumot- illetve új listát!
- A **ToString()** metódust egészítse ki az új értékekkel, melyeket az interfészből hozott!

Konténerosztály

A konténerosztály feladata kezelni egy adott településen található épületek listáját, és az ahhoz kapcsolódó metódusokat, lekérdezéseket.

TELEPULES

Hozza létre a **Telepules** osztályt, és végezze el az alábbi feladatokat!

Mezők

- Készítsen egy **Epulet** típusú objektumokat tárolni képes listát és inicializálja! Ne írjon hozzá propertyt!

Metódusok

- Készítsen metódust, melynek neve **Regisztral** és paraméterben vár egy **Epulet** típusú objektumot! Amennyiben a lista még nem tartalmazza az épületet, úgy adja hozzá a listához, ellenkező esetben dobjon kivételt a megfelelő üzenettel! **Extra:** Amennyiben implementálta az **ICollectionable** interfészt, rendezze a listát az alapértelmezett összehasonlítás alapján!

Propertyk

- Készítsen épületekkel visszatérő, csak getterrel rendelkező property-t, melynek neve **ModernLivingAltalEpitett100M2FolottiEpuletek**!
 - A lekérdezést Lambda-kifejezésekkel írja meg!
 - Gyűjtse ki a *Modern Living* nevű cég által épített 100 m^2 fölötti épületeket!
- Készítsen épületekkel visszatérő, csak getterrel rendelkező property-t, melynek neve **MegbizhatoCegAltalEpitettEpuletek**!
 - A lekérdezést Lambda-kifejezésekkel írja meg!
 - Gyűjtse ki a megbízható cégek által épített épületeket!
- Készítsen okosotthonokkal visszatérő, **OtLegkomfortosabbOkosotthon120m2FolottAmitEcoHomesEpitett** nevű csak getterrel rendelkező property-t!
 - A lekérdezést Lambda-kifejezésekkel írja meg!
 - Gyűjtse ki az öt legkomfortosabb, legalább 120 m^2 -es épületet, amit az *Eco Homes* épített! (Ehhez rendezze csökkenő sorrendbe az épületeket és a *Take()* utasítás segítségével csak az első öt elemet kérdezze le!)

Delegate-ek

- Hozzon létre egy **delegate**-et, mely **Okosotthon változós** és **egész értékű** függvényt reprezentál. (Ezt kihagyhatja, ha a következő feladatban **Func<>**-ot használ.)

- Készítsen valós értékkel visszatérő függvényt **Átlag** néven, mely a fenti delegate típusú függvényt kap paraméterben! Számítsa ki a gyűjteményben levő épületek ezen érték szerinti átlagát!
- A főprogramban jelenítse meg, mennyi az épületek átlagos területe!

Extra: Interfészek

Implementálja az **IEnumerable** interfészt, és térjen vissza a tárolás sorrendjében az összes épület klónjaival!

Főprogram

Töltse be a mellékelt **Program.cs** fájlt, és végezze el az alábbi feladatokat!

- Készítse fel a beolvasást arra, hogy esetleg rossz fájlnevet adtak meg!
- Készítse fel a példányosítást a **TeruletAlacsonyException** kivételre! Ebben az esetben hozzon létre új épületet, melynek a területe 50 m^2 !
- Készítse fel a példányosítást a **SzobakSzamaNemErvenyesException** kivételekre! Ezekben az esetekben Vizsgálja meg a rossz értéket! Ha az kisebb, mint 1, akkor állítsa 1-re! Ha nagyobb, mint 15, akkor állítsa 15-re!

További feladatok:

- Jelenítse meg az összes implementált lekérdezést a konténerosztályból!